

AssetMind

AI-Powered Industrial Asset Intelligence Platform

Project Name	AssetMind — AI-Powered Industrial Asset Intelligence Platform
Version	2.0.0
Team Name	Team Crew1.0
Authors	Tasneem Banu and KS Bande Nawaz Ahamed
Institution	Ballari Institute of Technology & Management (BITM)
Category	Hackathon Submission
Repository	https://github.com/tasneem38/AssetMind
Date	27 June 2026

Powered by FastAPI · React 19 · PostgreSQL · ChromaDB · Random Forest · XGBoost

Abstract

Industrial maintenance operations in manufacturing plants, utilities, and refineries generate vast quantities of operational data — inspection reports, work orders, incident records, and original equipment manufacturer (OEM) manuals. Despite this data abundance, maintenance engineers routinely face preventable equipment failures because critical information remains fragmented across disparate systems, unstructured documents, and institutional memory.

AssetMind is an AI-powered industrial asset intelligence platform that addresses this challenge by unifying four dimensions of operational data: structured relational maintenance records stored in PostgreSQL, unstructured OEM manual knowledge indexed in ChromaDB via semantic embeddings, machine learning models trained on NASA and UCI industrial datasets for failure prediction, and a conversational AI copilot powered by Sarvam AI's large language model. Together, these capabilities form a complete decision-support system for maintenance and reliability engineering teams.

The platform ingests nine OEM technical manuals for industrial equipment including Grundfos CR-series and NB/NBG pumps, Atlas Copco GA-30/55/75 compressors, and Siemens SIMOTICS SD motors. These documents are chunked using LangChain's RecursiveCharacterTextSplitter with 800-character chunks and 100-character overlap, encoded using the sentence-transformers all-MiniLM-L6-v2 model (384-dimensional embeddings), and persisted in a ChromaDB vector database for semantic retrieval.

The machine learning subsystem includes two distinct predictive models: a Random Forest classifier trained on the UCI AI4I 2020 Predictive Maintenance dataset (10,000 samples, 5 engineered features) that predicts equipment failure probability with risk classification, and an XGBoost regressor trained on the NASA CMAPSS FD001 turbofan engine dataset (20,631 training cycles, 14 selected sensor channels) for Remaining Useful Life (RUL) estimation.

The Knowledge Gap Detection algorithm identifies potentially preventable failures by correlating inspection findings (risk score ≥ 75 , follow-up required) against incident history for the same equipment. With 25 monitored assets, 500 work orders, 100 incidents, and 50 inspections, the platform computes preventable failure rates and ranks assets by composite risk score. AssetMind demonstrates that integrating RAG-augmented generation, SQL analytics, and ML prediction into a unified platform can meaningfully improve maintenance decision quality and prevent recurrent equipment failures.

Table of Contents

Abstract.....	2
Table of Contents.....	3
Chapter 1: Introduction.....	8
1.1 Background	8
1.2 Problem Statement.....	8
1.3 Motivation	8
1.4 Objectives.....	8
1.5 Scope	9
1.6 Existing Systems and Limitations	9
1.7 Proposed System and Advantages.....	9
Chapter 2: System Overview.....	10
2.1 Platform Architecture	10
2.2 Data Flow	10
Chapter 3: Technology Stack.....	12
3.1 Frontend Technologies	12
3.1.1 React 19	12
3.1.2 Vite 8.....	12
3.1.3 Tailwind CSS 4	12
3.1.4 Recharts, Axios, Framer Motion	12
3.2 Backend Technologies	12
3.2.1 FastAPI	12
3.2.2 SQLAlchemy + PostgreSQL 15.....	12
3.2.3 ChromaDB	12
3.2.4 Sentence Transformers (all-MiniLM-L6-v2)	13
3.2.5 Sarvam AI 105B.....	13
3.2.6 Pydantic v2	13
3.3 Machine Learning Technologies	13
3.3.1 scikit-learn RandomForestClassifier	13
3.3.2 XGBoost XGBRegressor.....	13
3.3.3 LangChain RecursiveCharacterTextSplitter.....	13
Chapter 4: System Architecture	15
4.1 Frontend Architecture	15
4.2 Backend Architecture.....	15
4.3 Database Architecture	15
4.4 RAG Architecture.....	15

4.5 ML Prediction Architecture.....	15
Chapter 5: Database Design	17
5.1 Schema Overview	17
5.2 Equipment Table	17
5.3 Work Order Table	17
5.4 Inspection Report Table.....	17
5.5 Incident Report Table	18
5.6 Entity-Relationship Diagram	19
Chapter 6: Backend Implementation	20
6.1 Application Entry Point (main.py).....	20
6.2 Dashboard Router (/dashboard)	20
GET /dashboard/.....	20
GET /dashboard/high-risk-assets	20
6.3 Equipment Router (/equipment).....	20
6.4 Insights Router (/insights)	21
GET /insights/knowledge-gaps.....	21
GET /insights/knowledge-gaps/summary	21
GET /insights/executive	21
6.5 Prediction Router (/predict).....	21
POST /predict/failure.....	21
POST /predict/rul	22
6.6 RAG Router (/ask)	22
Chapter 7: Frontend Implementation.....	23
7.1 Application Layout (AppLayout.jsx).....	23
7.2 Landing Page	23
7.3 Dashboard Page.....	23
7.4 Asset Explorer Page	23
7.5 Equipment Profile Page	23
7.6 Timeline Page	24
7.7 AI Copilot Page.....	24
7.8 Insights Page.....	24
7.9 Component Library Summary	24
Chapter 8: Retrieval-Augmented Generation (RAG)	25
8.1 Overview	25
8.2 OEM Manual Corpus	25
8.3 Ingestion Pipeline	25

8.4 Query-Time Retrieval: Simple RAG (POST /ask/)	26
8.5 Hybrid Retrieval: Copilot (POST /ask/copilot)	26
Chapter 9: AI Copilot	27
9.1 System Prompt Design	27
9.2 Equipment ID Detection	27
9.3 Confidence Score Formula	27
9.4 Risk Category Extraction	27
9.5 Root Cause Extraction	27
9.6 Recommended Actions Extraction	27
9.7 Complete Response Schema	28
Chapter 10: Knowledge Gap Detection	29
10.1 Algorithm Overview	29
10.2 Detection Criteria	29
10.3 Algorithm Flow	29
10.4 SQL Implementation	29
10.5 Business Value	30
Chapter 11: Machine Learning	31
11.1 Overview	31
11.2 AI4I Failure Prediction	31
11.2.1 Dataset	31
11.2.2 Training Configuration	31
11.2.3 Inference	31
11.3 CMAPSS RUL Prediction	31
11.3.1 Dataset	31
11.3.2 Feature Engineering	32
11.3.3 Training Configuration	32
11.3.4 Health Score Mapping	32
11.4 Model File Summary	32
Chapter 12: API Documentation	33
12.1 Complete Endpoint Reference	33
12.2 Example Requests and Responses	34
POST /ask/copilot — Request Body	34
POST /ask/copilot — Response (abbreviated)	34
POST /predict/failure — Request Body	34
POST /predict/failure — Response	34
POST /predict/rul — Request Body	34

POST /predict/rul — Response	34
12.3 Error Response Codes	34
Chapter 13: Application Workflow	35
13.1 End-to-End User Journey	35
Chapter 14: Folder Structure	36
14.1 Complete Project Directory Tree	36
Chapter 15: Screenshots	38
15.1 Operations Dashboard	38
15.2 Asset Explorer	39
15.3 Equipment Profile	39
15.4 Timeline View	40
15.5 AI Copilot	40
15.6 ML Prediction Panel	41
15.7 Insights & Analytics	41
Chapter 16: Challenges Faced	42
16.1 ChromaDB API Version Compatibility	42
16.2 Sarvam AI SDK Key Mapping	42
16.3 Empty LLM Response Handling	42
16.4 CMAPSS RUL Target Engineering	42
16.5 AI4I Class Imbalance	42
16.6 RAG Context Window Management	42
16.7 Equipment ID Regex Precision	43
16.8 Frontend Multi-Call State Management	43
Chapter 17: Future Enhancements	44
17.1 IoT Real-Time Sensor Integration (MQTT / OPC-UA)	44
17.2 Digital Twin Integration	44
17.3 SAP Plant Maintenance Integration	44
17.4 Multi-Agent Maintenance Orchestration	44
17.5 Edge AI Deployment	44
17.6 Predictive Maintenance Scheduling	44
17.7 Authentication and Multi-Tenant Support	44
17.8 Voice Interface for Shop Floor Use	45
17.9 Expanded OEM Knowledge Base	45
Chapter 18: Conclusion	46
References	47
Appendix A: Installation Guide	48

A.1 Prerequisites48

A.2 Backend Setup.....48

A.3 Frontend Setup48

Appendix B: Environment Variables49

Appendix C: Data Asset Summary50

Appendix D: API Quick Reference51

Chapter 1: Introduction

1.1 Background

The industrial maintenance sector is undergoing a fundamental transformation driven by advances in artificial intelligence, large language models, and vector database technologies. Traditional Computerised Maintenance Management Systems (CMMS) excel at storing maintenance records but provide no intelligent reasoning over their data. Engineers must manually correlate inspection findings with OEM manual recommendations, cross-reference incident histories, and exercise individual expertise to diagnose complex failure modes.

The consequences of inadequate maintenance intelligence are severe. Unplanned equipment downtime in process industries costs an estimated USD 50 billion annually, with 42% of unplanned downtime attributed to failures that could have been predicted or prevented. In safety-critical industries such as power generation, chemicals, and pharmaceuticals, equipment failures carry not only economic cost but significant operational risk.

1.2 Problem Statement

Industrial organisations face five critical challenges in equipment maintenance:

- Maintenance history is fragmented across multiple systems (CMMS, spreadsheets, paper records, emails), making pattern recognition difficult.
- Engineers spend excessive time manually searching hundreds of pages of OEM technical manuals to diagnose unfamiliar failure modes.
- Inspection recommendations are frequently not acted upon, creating knowledge gaps that lead to preventable failures.
- Existing CMMS solutions are passive data stores — they record events but provide no predictive intelligence or AI-driven recommendations.
- There is no single platform combining structured maintenance data, unstructured document knowledge, and predictive ML in a unified decision-support interface.

1.3 Motivation

The primary motivation for AssetMind is to democratise maintenance intelligence — to put the combined knowledge of engineering manuals, historical failure patterns, and predictive models into the hands of every maintenance engineer through a natural-language conversational interface. A junior engineer should be able to ask 'Why is pump PMP-CW-101 failing?' and receive a structured, evidence-backed response that cites both OEM manual recommendations and the asset's actual maintenance history.

1.4 Objectives

1. Design and implement a unified industrial asset intelligence platform integrating relational data, vector search, and generative AI.
2. Build a RAG (Retrieval-Augmented Generation) pipeline over real OEM technical manuals for equipment-specific knowledge retrieval.

3. Develop a Knowledge Gap Detection algorithm that correlates inspection recommendations with incident history to surface preventable failures.
4. Train and deploy Random Forest and XGBoost predictive models for failure probability and RUL estimation.
5. Implement an AI Copilot that combines relational database context with manual knowledge to generate evidence-backed maintenance recommendations.
6. Create a professional React dashboard with real-time PostgreSQL integration, risk scoring, and fleet-level analytics.

1.5 Scope

AssetMind version 2.0 covers 25 industrial assets across 9 equipment types (pumps, motors, compressors, heat exchangers, valves, cooling towers, conveyors), 500 synthetic work orders, 100 incident reports, and 50 inspection reports. The knowledge base spans 9 OEM technical manuals. Two ML models and a conversational AI Copilot powered by Sarvam AI 105B are fully deployed.

1.6 Existing Systems and Limitations

Current industrial maintenance solutions including IBM Maximo, SAP Plant Maintenance, UpKeep, and Fiix effectively manage work orders and asset registers but share key limitations: no natural-language query interface, no built-in RAG capability over OEM documentation, limited or no accessible ML-based predictive analytics, no automated knowledge gap detection, and high implementation cost limiting adoption.

1.7 Proposed System and Advantages

AssetMind proposes a lightweight, AI-first alternative that delivers a natural language copilot interface (Sarvam AI 105B with RAG grounding), automated knowledge gap detection, dual ML prediction (failure probability and RUL), full source citation audit trails, a modern React SPA with teal design system, and Docker-based deployment with clear tier separation.

Chapter 2: System Overview

2.1 Platform Architecture

AssetMind is a full-stack industrial intelligence platform built on a layered architecture. The React Single Page Application communicates with a FastAPI backend over HTTP/REST. The backend orchestrates four data sources: PostgreSQL for structured maintenance records, ChromaDB for vector-indexed OEM manual knowledge, serialised scikit-learn and XGBoost models for ML inference, and the Sarvam AI external LLM API for natural language generation.

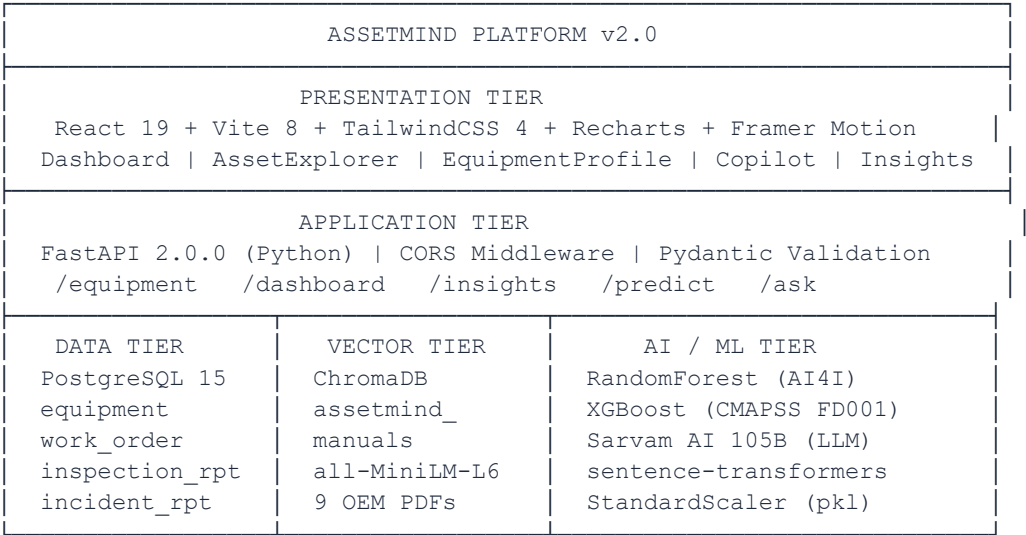
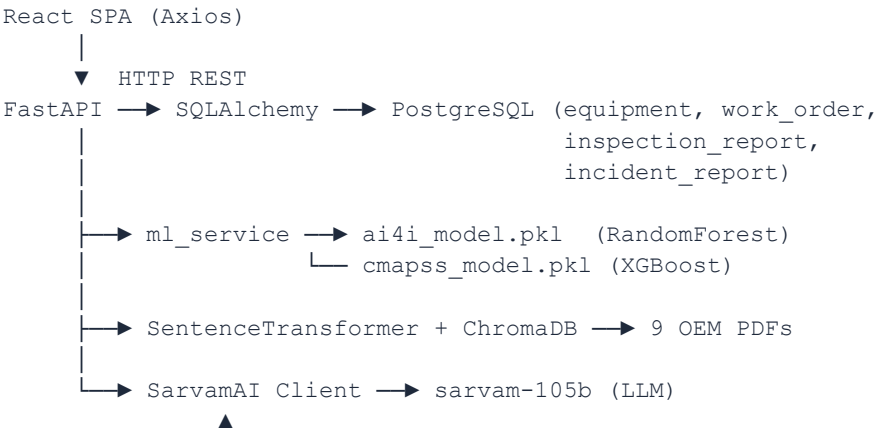


Figure 2.1 – AssetMind Three-Tier Architecture

2.2 Data Flow

User interactions in the React frontend trigger Axios HTTP requests to the FastAPI backend. The backend routes each request to the appropriate service layer: PostgreSQL queries via SQLAlchemy for structured data, ChromaDB semantic search for manual knowledge, ML model inference via ml_service.py, or the Sarvam AI API for LLM generation. All AI responses are grounded in retrieved evidence and returned with structured metadata including confidence scores, risk categories, and cited sources.



RAG context + system prompt

Figure 2.2 – Component Data Flow Diagram

Chapter 3: Technology Stack

3.1 Frontend Technologies

3.1.1 React 19

React 19 serves as the primary UI framework. The application is structured as a Single Page Application with React Router DOM v7 for client-side routing across six application pages (Dashboard, Asset Explorer, Equipment Profile, Timeline, Copilot, Insights) and a public 7-section Landing page. React's component model enables reusable UI elements: StatsCard, RiskBadge, KnowledgeGapCard, ChatWindow, TimelineView, AssetSearch.

3.1.2 Vite 8

Vite 8 replaces Create React App as the build toolchain. Its native ES module-based development server provides near-instant HMR (Hot Module Replacement). The production build uses tree-shaking and code splitting. The `@tailwindcss/vite` plugin is configured in `vite.config.js` for zero-runtime Tailwind compilation.

3.1.3 Tailwind CSS 4

Tailwind CSS v4 is used for all styling. The design system is defined in `src/styles/global.css` using CSS custom properties: teal primary (#0F766E), light primary (#14B8A6), dark navy sidebar (#0D1B2A), light background (#F0F4F8). Component classes (`.card`, `.btn`, `.tag`, `.stat-card`) are defined in `@layer` components.

3.1.4 Recharts, Axios, Framer Motion

Recharts v3.8 provides AreaChart visualisations on Dashboard and Insights pages with teal gradient fills. Axios v1.18 handles all HTTP communication via a centralised `api.js` client with baseURL `http://127.0.0.1:8000`. Framer Motion v12 provides animation capabilities for page transitions and component entrances.

3.2 Backend Technologies

3.2.1 FastAPI

FastAPI (version 2.0.0 of the AssetMind API, defined in `backend/app/main.py`) registers five routers and configures CORS middleware allowing requests from the Vite dev server (`localhost:5173`). FastAPI's automatic OpenAPI documentation, Pydantic-based request validation, and async-first architecture make it suitable for an AI platform with mixed workloads.

3.2.2 SQLAlchemy + PostgreSQL 15

SQLAlchemy is used as the database toolkit with a synchronous engine connected to PostgreSQL at the configured `DATABASE_URL`. Queries use SQLAlchemy's `text()` function with parameterised placeholders for SQL injection prevention. The `SessionLocal` factory provides session-scoped operations; direct `engine.connect()` context managers handle aggregate queries.

3.2.3 ChromaDB

ChromaDB is the vector database for RAG. A PersistentClient stores the index at backend/chroma_db/. The collection assetmind_manuals contains chunked text from 9 OEM PDFs, each encoded as a 384-dimensional all-MiniLM-L6-v2 embedding. At query time, top-5 chunks (/ask/) or top-8 chunks (/ask/copilot) are retrieved by cosine similarity.

3.2.4 Sentence Transformers (all-MiniLM-L6-v2)

The all-MiniLM-L6-v2 model (22M parameters, 384-dimensional output) generates dense vector embeddings during both ingestion and retrieval. It was selected for its balance of encoding speed and semantic quality on sentence similarity benchmarks. The same model instance serves both document embedding and query embedding.

3.2.5 Sarvam AI 105B

The sarvamai Python SDK connects to the Sarvam AI API using the SARVAM_API_KEY environment variable. The sarvam-105b model powers both the simple /ask/ endpoint (manual RAG only) and the full /ask/copilot endpoint (hybrid RAG + relational + LLM). A compatibility shim in backend/main.py maps SARVAM_API_KEY to GEMINI_API_KEY for underlying library compatibility.

3.2.6 Pydantic v2

Pydantic provides data validation for all API schemas. FailureInput validates five numeric sensor fields. RULInput validates a 14-element sensor array with explicit min/max length constraints. FailureOutput and RULOutput enforce ML response structure.

3.3 Machine Learning Technologies

3.3.1 scikit-learn RandomForestClassifier

200 estimators, max_depth=10, class_weight='balanced', random_state=42. The balanced weighting addresses the AI4I dataset's 3.4% failure rate class imbalance. Trained on the UCI AI4I 2020 dataset using an 80/20 stratified split.

3.3.2 XGBoost XGBRegressor

300 estimators, max_depth=6, learning_rate=0.05, subsample=0.8, colsample_bytree=0.8, random_state=42. Trained on NASA CMAPSS FD001 with a piecewise-linear RUL target clipped at MAX_RUL=130 cycles to focus learning on the degradation phase.

3.3.3 LangChain RecursiveCharacterTextSplitter

Used exclusively for document chunking during RAG ingestion. chunk_size=800, chunk_overlap=100. The recursive strategy first splits on paragraph boundaries, then sentences, then words, preserving semantic coherence within chunks.

Technology	Version	Purpose	Tier
React	19.2.6	SPA UI Framework	Frontend
Vite	8.0.12	Build Tool + Dev Server	Frontend
Tailwind CSS	4.3.1	Utility-first Styling	Frontend
Recharts	3.8.1	Data Visualisation	Frontend

Technology	Version	Purpose	Tier
Axios	1.18.1	HTTP Client	Frontend
React Router DOM	7.18.0	Client-side Routing	Frontend
Framer Motion	12.40.0	Animations	Frontend
FastAPI	Latest	REST API Framework	Backend
SQLAlchemy	Latest	ORM / SQL Toolkit	Backend
Pydantic	v2	Request / Response Validation	Backend
PostgreSQL	15	Relational Database	Database
ChromaDB	Latest	Vector Database	AI/ML
Sentence Transformers	Latest	all-MiniLM-L6-v2 Embeddings	AI/ML
Sarvam AI 105B	API	Large Language Model	AI/ML
scikit-learn	Latest	Random Forest Classifier	ML
XGBoost	Latest	RUL Regressor	ML
LangChain	Latest	Text Chunking	AI/ML
pypdf	Latest	PDF Text Extraction	Ingestion

Table 3.1 – Complete Technology Stack

Chapter 4: System Architecture

4.1 Frontend Architecture

The React frontend is organised around a shell layout component (AppLayout.jsx) rendering a fixed 240px dark navy Sidebar and top bar, with page content via React Router's Outlet. Public routes (Landing, Login) render outside AppLayout. State management is entirely local using React useState and useEffect — no Redux or global state library.

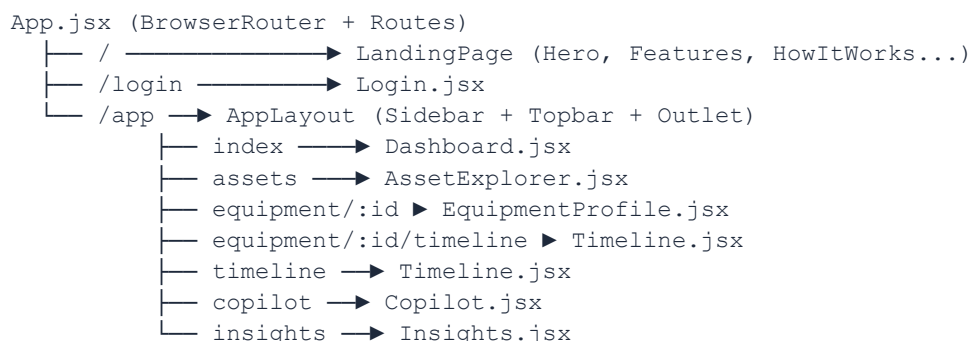


Figure 4.1 – React Router Page Hierarchy

4.2 Backend Architecture

The FastAPI application registers five routers at startup: equipment_router (/equipment), rag_router (/ask), insights_router (/insights), dashboard_router (/dashboard), and predict_router (/predict). CORS middleware permits requests from the Vite dev server. Two service modules handle business logic: postgres_service.py for structured database operations and ml_service.py for model inference. The equipment_parser.py utility provides regex-based equipment ID extraction.

4.3 Database Architecture

PostgreSQL stores four tables linked by equipment_id as a natural key: equipment (25 rows), work_order (500 rows), inspection_report (50 rows), incident_report (100 rows). The SessionLocal factory manages session-scoped operations; direct engine.connect() context managers handle read-heavy aggregate queries.

4.4 RAG Architecture

The RAG pipeline operates in two phases. During ingestion, PDF manuals are parsed page by page with pypdf, split into 800-character chunks, embedded with all-MiniLM-L6-v2, and persisted in ChromaDB with metadata (manual name, page number). During inference, the user's question is encoded, ChromaDB returns top-k similar chunks by cosine similarity, and the retrieved text forms the OEM Manual Evidence section of the LLM prompt.

4.5 ML Prediction Architecture

Two independent ML pipelines serve prediction requests. The AI4I failure pipeline accepts 5 scalar sensor inputs, applies StandardScaler normalisation, and calls RandomForestClassifier.predict_proba() for failure probability. The CMAPSS RUL pipeline

accepts a 14-element sensor array, applies StandardScaler, and calls XGBRegressor.predict() for RUL in cycles, which is mapped to a 0-100 health score. Both models are lazy-loaded on first request and cached as module-level globals.

Chapter 5: Database Design

5.1 Schema Overview

The AssetMind database consists of four tables sharing a natural key (`equipment_id`) linking all operational records to their originating asset. The schema captures the complete maintenance lifecycle: equipment registration, periodic inspection findings, corrective work orders, and post-failure incident reports.

5.2 Equipment Table

Column	Type	Constraints	Description
<code>equipment_id</code>	VARCHAR	PRIMARY KEY	Unique asset identifier (e.g. PMP-CW-101)
<code>equipment_name</code>	VARCHAR	NOT NULL	Human-readable name (e.g. Cooling Water Pump A)
<code>equipment_type</code>	VARCHAR	NOT NULL	Type: Pump, Motor, Compressor, Heat Exchanger, etc.
<code>location</code>	VARCHAR	NOT NULL	Physical plant location
<code>manufacturer</code>	VARCHAR	NOT NULL	OEM name (Kirloskar Brothers, KSB Pumps, etc.)
<code>criticality</code>	VARCHAR	NOT NULL	Critical / High / Medium / Low
<code>install_date</code>	DATE	NOT NULL	Equipment installation date

Table 5.1 – equipment table (25 rows)

The 25 assets span 9 equipment types across 4 locations: Cooling Water Plant, Utility Area, Process Area, and HVAC Area. Equipment IDs follow the convention: type-prefix + location-code + sequence (e.g. PMP-CW-101, MTR-CW-102, CMP-AIR-201, HEX-PR-201, VLV-CW-101).

5.3 Work Order Table

Column	Type	Constraints	Description
<code>wo_id</code>	VARCHAR	PRIMARY KEY	Work order ID (WO-0001 to WO-0500)
<code>equipment_id</code>	VARCHAR	FK → equipment	Asset the work order is raised for
<code>date</code>	DATE	NOT NULL	Work order date
<code>problem</code>	TEXT	NOT NULL	Reported problem description
<code>root_cause</code>	TEXT	NOT NULL	Diagnosed root cause
<code>action_taken</code>	TEXT	NOT NULL	Corrective action performed
<code>technician</code>	VARCHAR	NOT NULL	Technician name
<code>status</code>	VARCHAR	NOT NULL	Completed / Closed / Open

Table 5.2 – work_order table (500 rows)

5.4 Inspection Report Table

Column	Type	Constraints	Description
inspection_id	VARCHAR	PRIMARY KEY	Inspection ID (INSP-0001 to INSP-0050)
equipment_id	VARCHAR	FK → equipment	Inspected asset
inspection_date	DATE	NOT NULL	Date of inspection
inspector	VARCHAR	NOT NULL	Inspector name (R Sharma, A Kumar, etc.)
vibration_reading	NUMERIC	NOT NULL	Vibration level (mm/s RMS)
temperature_reading	NUMERIC	NOT NULL	Surface temperature (degrees C)
risk_score	INTEGER	NOT NULL	Calculated risk score 0-100
finding	TEXT	NOT NULL	Inspection finding description
recommendation	TEXT	NOT NULL	Recommended corrective action
follow_up_required	VARCHAR	NOT NULL	Yes / No — whether follow-up action is required

Table 5.3 – inspection_report table (50 rows)

5.5 Incident Report Table

Column	Type	Constraints	Description
incident_id	VARCHAR	PRIMARY KEY	Incident ID (INC-0001 to INC-0100)
equipment_id	VARCHAR	FK → equipment	Asset where failure occurred
incident_date	DATE	NOT NULL	Date of incident
failure_mode	VARCHAR	NOT NULL	Failure mode (Cavitation, Bearing Failure, etc.)
root_cause	TEXT	NOT NULL	Diagnosed root cause
contributing_factors	TEXT	NOT NULL	Secondary contributing factors
corrective_action	TEXT	NOT NULL	Corrective action taken
preventable	VARCHAR	NOT NULL	Yes / No / Partially — preventability assessment
linked_work_order	VARCHAR	FK → work_order	Associated work order ID

Table 5.4 – incident_report table (100 rows)

5.6 Entity-Relationship Diagram

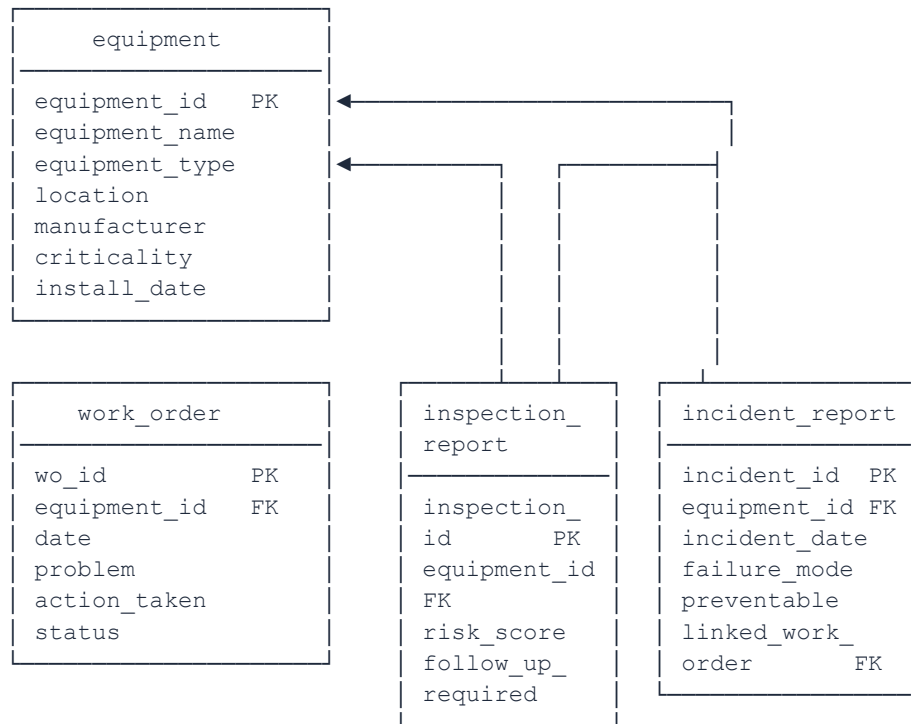


Figure 5.1 – Entity-Relationship Diagram (all FK relationships shown)

Chapter 6: Backend Implementation

6.1 Application Entry Point (main.py)

The FastAPI application initialised in backend/app/main.py registers five routers and configures CORS middleware for the Vite dev server (localhost:5173) and localhost:3000. The root GET / endpoint returns a health-check payload confirming API version 2.0.0. The backend/main.py wrapper maps the SARVAM_API_KEY environment variable to GEMINI_API_KEY on process startup for SDK compatibility.

6.2 Dashboard Router (/dashboard)

GET /dashboard/

Returns aggregate KPI statistics. The postgres_service.get_dashboard_stats() function executes six SQL queries: COUNTs of equipment, work orders, incidents, and inspections; a knowledge gap count (inspections with follow_up='Yes', risk_score>=75, and the equipment has incidents); a preventable incident count (preventable IN ('Yes','Partially')); the top failure mode by frequency; and a count of high-risk assets using the composite risk score formula (score >= 70).

KPI Field	SQL Logic	Example
total_assets	COUNT(*) FROM equipment	25
total_work_orders	COUNT(*) FROM work_order	500
total_incidents	COUNT(*) FROM incident_report	100
total_inspections	COUNT(*) FROM inspection_report	50
high_risk_assets	Assets with composite risk score >= 70	8
knowledge_gaps	follow_up=Yes + risk>=75 + has incidents	12
preventable_failures	preventable IN ('Yes','Partially')	75
top_failure_mode	GROUP BY failure_mode ORDER BY COUNT DESC LIMIT 1	Bearing Failure

Table 6.1 – Dashboard KPI Response Fields

GET /dashboard/high-risk-assets

Returns all 25 equipment records ranked by composite risk score: $\text{risk_score} = (\text{incident_count} \times 0.4) + (\text{avg_inspection_risk} \times 0.4) + (\text{open_followups} \times 0.2)$, capped at 100. Risk levels: Critical (>=90), High (>=70), Medium (>=40), Low (<40). A single SQL query with three LEFT JOINS against subqueries computes this in one database round-trip.

6.3 Equipment Router (/equipment)

Endpoint	Method	Description
GET /equipment/	GET	All 25 equipment records from the equipment table
GET /equipment/{id}	GET	Single equipment record by equipment_id

Endpoint	Method	Description
GET /equipment/{id}/briefing	GET	Aggregated summary: criticality, WO count, incident count, last inspection date, open followups, most common failure mode
GET /equipment/{id}/timeline	GET	Chronologically sorted events from all three history tables: Inspection, Recommendation, Work Order, Repair, Incident
GET /equipment/{id}/incidents	GET	Incident reports for equipment, ordered by incident_date DESC
GET /equipment/{id}/inspections	GET	Inspection reports for equipment, ordered by inspection_date DESC
GET /equipment/{id}/health	GET	Health score (0-100) and status string: Healthy / At Risk / Critical

Table 6.2 – Equipment Router Endpoints

The health score uses a penalty model: $\text{health_score} = 100 - \min(\text{incident_count} \times 5, 40) - \min((\text{avg_risk} / 100) \times 30, 30) - \min(\text{open_followups} \times 3, 30)$. Maximum penalties: 40 for incident history, 30 for inspection risk, 30 for unresolved followups. Status: Healthy (≥ 80), At Risk (≥ 50), Critical (< 50).

6.4 Insights Router (/insights)

GET /insights/knowledge-gaps

Iterates over all inspection records ordered by risk_score DESC. For each inspection where follow_up_required='Yes' AND risk_score ≥ 75 , it queries the incident count for that equipment. Qualifying inspections are flagged as 'Potentially Preventable Failure' and enriched with the equipment's most common failure_mode, incident count, and risk classification (Critical ≥ 85 , High ≥ 75 , Medium otherwise).

GET /insights/knowledge-gaps/summary

Returns total gap count, preventable_failure_rate ($\text{gap_count} / \text{total_incidents} \times 100$), and top 5 assets ranked by number of gap inspections. Uses a single optimised SQL query with a correlated EXISTS subquery.

GET /insights/executive

Generates executive summary from real data: highest-risk asset (composite score), most common failure mode, preventable failure count, fleet average risk score, and a type-specific recommended action string derived from a type_actions dictionary keyed by equipment_type (Pump, Motor, Compressor, Heat Exchanger, Valve).

6.5 Prediction Router (/predict)

POST /predict/failure

Accepts five sensor values (air_temperature, process_temperature, rotational_speed, torque, tool_wear). ml_service.predict_failure() loads AI4I models lazily, scales inputs with saved StandardScaler, calls predict_proba() on RandomForestClassifier. Returns failure_probability

(0.0-1.0), risk_level (Low/Medium/High with thresholds 0.4/0.7), and top_features (top 3 by feature_importances_).

POST /predict/rul

Accepts 14 sensor_values in CMAPSS sensor order. ml_service.predict_rul() loads CMAPSS models lazily, scales inputs, calls XGBRegressor.predict(). Maps predicted cycles to health_score (0-100) using max_rul=400. Returns remaining_useful_life (cycles) and degradation_trend (Healthy/Stable/Declining/Critical at thresholds 300/200/100 cycles).

6.6 RAG Router (/ask)

POST /ask/ performs simple manual RAG: encodes question with SentenceTransformer, queries ChromaDB for top-5 chunks, assembles a 4,000-character context, calls Sarvam 105B with a manual-only prompt. POST /ask/copilot performs hybrid RAG: extracts equipment ID with regex, queries PostgreSQL for incidents/inspections/work orders, retrieves top-8 ChromaDB chunks, assembles a 12,000-character combined context, calls Sarvam 105B with the full system prompt. Detailed description in Chapters 8 and 9.

Chapter 7: Frontend Implementation

7.1 Application Layout (AppLayout.jsx)

AppLayout establishes the two-panel shell for all authenticated pages. A fixed-width 240px dark navy Sidebar (#0D1B2A) anchors the left edge with the AssetMind brand logo (SVG) and navigation links. The main content area (margin-left: 240px) renders the Topbar and page content via React Router's Outlet. Fonts Inter (body) and JetBrains Mono (code/IDs) are loaded from Google Fonts.

7.2 Landing Page

The Landing page (src/pages/Landing/index.jsx) composes seven independent section components: Hero (full-viewport gradient banner with 'Launch App' CTA), Problem (pain points grid), Solution (AssetMind's approach), Features (six key capabilities), HowItWorks (3-step process), Metrics (fleet-level KPIs), Preview (app screenshot mockup), and a CTA section with DemoRequest form. The landing renders outside AppLayout at the root URL /.

7.3 Dashboard Page

On mount, Dashboard fires three concurrent API calls via Promise.all: getDashboardStats(), getHighRiskAssets(), and getKnowledgeGapsSummary(). Skeleton placeholder components animate during loading. The page renders: four KPI StatsCards (Total Assets, Open Work Orders, High Risk Assets, Incidents), a Recharts AreaChart (Fleet Summary), a custom SVG donut chart (Risk Distribution, drawn with raw path elements and calculated risk_level percentages), a High Risk Assets table (top 8, clickable rows navigate to EquipmentProfile), and Knowledge Gap Alert cards.

7.4 Asset Explorer Page

Fetches all 25 equipment records via getEquipment() and renders them in a 3-column card grid. Each card shows the equipment_id in teal monospace, equipment_type, a RiskBadge, location, and manufacturer. An AssetSearch controlled input filters cards in real time by equipment_id, type, or location. A TYPE_ICONS dictionary maps equipment types to decorative emoji watermarks. Clicking any card navigates to /app/equipment/{id}.

7.5 Equipment Profile Page

The most data-rich page. Retrieves six endpoints on mount: getEquipmentById, getEquipmentHealth, getBriefing, getEquipmentIncidents, getEquipmentInspections. The embedded PredictPanel sub-component provides interactive ML prediction for both models. Page sections: asset header (equipment_id, type, location, install date, manufacturer, criticality badge), health score gauge (numeric + status colour), briefing summary panel, incidents table, inspections table, and PredictPanel.

The PredictPanel renders two form sections: AI4I failure prediction (5 numeric inputs pre-populated with realistic defaults: air_temperature 298.1K, rotational_speed 1551 RPM, torque 42.8 Nm, tool_wear 0 min) and CMAPSS RUL prediction (14 sensor inputs with physical labels and units from RUL_SENSOR_LABELS array). Prediction results appear inline with coloured risk indicators.

7.6 Timeline Page

Displays equipment maintenance history as a vertical chronological sequence using the TimelineView component. Each event is typed (Inspection, Recommendation, Work Order, Repair, Incident) and rendered with a coloured left-border indicator. Events are sorted ascending by date. The page requires an equipment_id parameter, either from the URL (/equipment/:id/timeline) or selected interactively.

7.7 AI Copilot Page

Implements a full-featured chat interface. ChatWindow renders message history with alternating user/assistant styles. Assistant messages are structured as rich cards: cause (primary root cause), evidence (bullet list of actions or citations), OEM reference (manual name + page), confidence score (0-100%), and risk level badge. Five example questions pre-loaded in the sidebar trigger immediate API calls on click. The mapResponseToMessage function parses the backend's structured JSON into the ChatWindow card format using regex extraction of markdown sections.

7.8 Insights Page

Provides fleet-level analytics via four concurrent API calls: getExecutiveInsights, getKnowledgeGapsSummary, getHighRiskAssets, and getKnowledgeGaps. Renders: an executive insights card (highest risk asset, most common failure mode, preventable failure count, recommended action), a risk trend AreaChart, a failure mode frequency bar chart derived from knowledge gap data (aggregated from gaps[].common_failure_mode), and a predicted failure list (top Critical/High risk assets).

7.9 Component Library Summary

Component	File	Purpose
AppLayout	components/AppLayout.jsx	Shell: Sidebar + Topbar + Outlet
Sidebar	components/Sidebar.jsx	Nav links, brand logo, AI Engine status indicator
Topbar	components/Topbar.jsx	Top bar with page context
StatsCard	components/StatsCard.jsx	KPI metric card with icon, value, trend, colour accent
RiskBadge	components/RiskBadge.jsx	Colour-coded pill: Critical/High/Medium/Low
KnowledgeGapCard	components/KnowledgeGapCard.jsx	Knowledge gap alert card with finding + recommendation
ChatWindow	components/ChatWindow.jsx	Copilot message renderer with structured assistant card
AssetSearch	components/AssetSearch.jsx	Controlled search input for asset filtering
TimelineView	components/TimelineView.jsx	Chronological event list with coloured type indicators

Table 7.1 – Reusable Component Library

Chapter 8: Retrieval-Augmented Generation (RAG)

8.1 Overview

The RAG subsystem enables AssetMind to answer equipment-specific questions grounded in actual OEM technical documentation rather than relying solely on LLM parametric knowledge. This is critical for industrial applications where equipment-specific torque specifications, maintenance intervals, and troubleshooting procedures must be accurate and traceable to authoritative sources.

8.2 OEM Manual Corpus

Manual File	Equipment Family	Coverage
1_CR.pdf	Grundfos CR Series	Centrifugal multistage pump installation, operation
1_NB NBG.pdf	Grundfos NB/NBG	End-suction pump service manual
98419736_02.2023_CR_IO_US.pdf	Grundfos CR/CRI/CRN/CRT	Installation and operation manual (US, 2023)
98882912_06.2026_NB_NBG_MTB_IOS.pdf	Grundfos NB/NBG/MTB	International operations manual (2026)
A5E41126458A_Operating_Instructions_SIMOTICS.pdf	Siemens SIMOTICS SD	Motor operating instructions
LVStdMotorsManual_EN_01_2009.pdf	Siemens LV Standard	Low-voltage motor service manual
Operating_Instructions_SIMOTICS_SD_en.pdf	Siemens SIMOTICS SD	Motor operating instructions (alternate edition)
atlas-copco-ga-30-manual.pdf	Atlas Copco GA-30	Air compressor maintenance manual
atlas-copco-ga-55-75.pdf	Atlas Copco GA-55/75	Air compressor installation and service manual

Table 8.1 – OEM Manual Corpus (9 PDFs)

8.3 Ingestion Pipeline

The ingestion pipeline (`rag/ingestion_pipeline.py`) is run once as a setup step. It follows a four-stage process:

7. PDF Discovery: `pathlib.Path.glob('*.*pdf')` scans the manuals directory.
8. Page Extraction: `pypdf.PdfReader` opens each PDF; `page.extract_text()` extracts text page by page. Pages with no extractable text are skipped.
9. Text Chunking: LangChain's `RecursiveCharacterTextSplitter` splits each page's text (`chunk_size=800`, `chunk_overlap=100`) attempting paragraph > sentence > word boundaries.

10. Embedding and Storage: Each chunk is encoded with `SentenceTransformer.encode()` to a 384-dimensional vector and stored in ChromaDB with `chunk_id` (`{pdf_stem}_p{page}_c{chunk_idx}`), document text, embedding, and metadata (`{manual: stem, page: int}`).

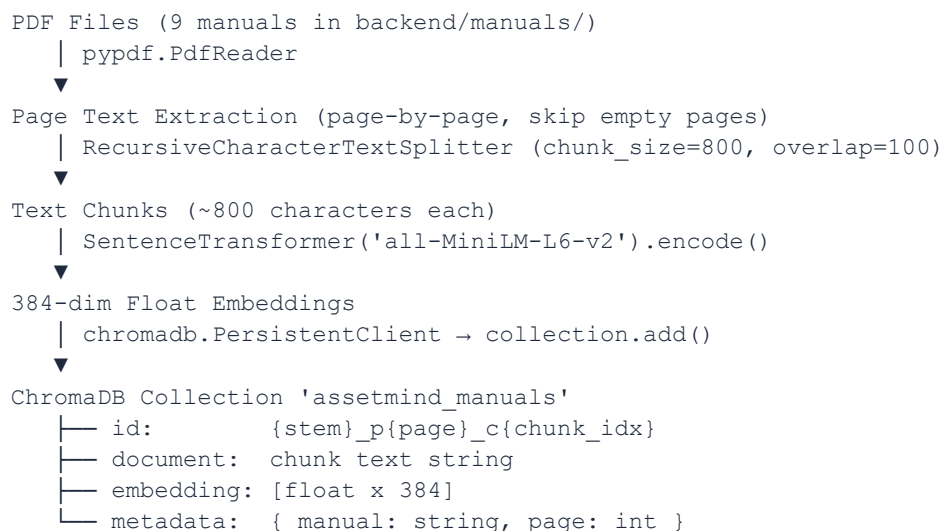


Figure 8.1 – RAG Ingestion Pipeline

8.4 Query-Time Retrieval: Simple RAG (POST /ask/)

(1) User question is encoded with `SentenceTransformer.encode()` to a 384-dim query vector. (2) `collection.query()` performs cosine similarity search, returning top-5 chunks with documents and metadata. (3) Chunks concatenated to a 4,000-character context. (4) A structured prompt requests Summary, Likely Causes, Operational Risk, Recommended Actions, Sources from the LLM. (5) Sarvam 105B called via `sarvam_client.chat.completions(model='sarvam-105b')`. (6) Response returns question, answer, and sources array (manual + page).

8.5 Hybrid Retrieval: Copilot (POST /ask/copilot)

Before querying ChromaDB, `extract_id()` attempts to find an equipment ID in the question (regex: alphanumeric + hyphen, >= 3 chars, at least one digit). If found, `postgres_service` retrieves incidents, inspections, and work orders for that equipment (5 records each, date DESC). ChromaDB returns top-8 chunks. The combined context (relational data + manual evidence, capped at 12,000 characters) is assembled and passed with the full `SYSTEM_PROMPT` to Sarvam 105B.

Chapter 9: AI Copilot

9.1 System Prompt Design

The system prompt defines the Copilot's identity as 'AssetMind Copilot, an AI assistant for industrial maintenance and reliability engineering.' It specifies eight knowledge sources (equipment history, work orders, inspection reports, incident reports, OEM manuals, SOPs, failure risk scores, asset health scores), instructs the model to prioritise operational evidence and cite OEM recommendations, and enforces a fixed output format with five markdown sections.

The critical grounding constraint in the system prompt is: 'Never invent incidents, inspections, work orders, or manual references.' This prevents the LLM from hallucinating maintenance records not present in the retrieved context. The output format enforces structured reasoning: **### Summary, ### Evidence, ### Risk Assessment, ### Recommendation, ### Sources**.

9.2 Equipment ID Detection

The `equipment_parser.py` service uses the regex pattern `(?i)\b(?=[A-Z0-9-]*\d)[A-Z0-9-]{3,}\b`. The lookahead `(?=[A-Z0-9-]*\d)` requires at least one digit; the minimum length of 3 prevents false matches on short tokens. The match is returned in uppercase. This reliably detects IDs like PMP-CW-101, MTR-CW-102, CMP-AIR-201, HEX-PR-201, VLV-CW-101 in free-text questions.

9.3 Confidence Score Formula

```
confidence = min(0.95,
    0.50                                # base confidence
    + 0.05 * incident_count             # +5% per incident record found
    + 0.03 * inspection_count           # +3% per inspection record found
    + 0.02 * manual_citation_count      # +2% per manual chunk retrieved
)
```

This formula rewards answers grounded in more evidence. A response supported by 5 incidents (0.25), 5 inspections (0.15), and 8 manual chunks (0.16) yields $\text{confidence} = \min(0.95, 0.50 + 0.56) = 0.95$. The 0.95 cap acknowledges that no AI response should claim absolute certainty.

9.4 Risk Category Extraction

The `risk_category` field is derived by scanning the lowercase LLM answer for indicator keywords: 'critical' or 'immediate' -> Critical; 'high' or 'urgent' -> High; 'medium' or 'moderate' -> Medium; otherwise Low. This provides a structured machine-readable risk signal independent of the narrative prose.

9.5 Root Cause Extraction

A curated taxonomy of 13 industrial root causes is checked against the lowercase LLM response: Bearing Wear, Bearing Failure, Cavitation, Corrosion, Overheating, Seal Leakage, Vibration, Misalignment, Fouling, Lubrication Failure, Overload, Insulation Failure, Pitting. Matches are returned in `root_causes`. If no taxonomy match is found, the fallback uses the top 3 `failure_mode` values from the equipment's incident history in the relational data.

9.6 Recommended Actions Extraction

The regex `r'###\s*Recom[a-z ]*\n(?:.*?)(?=###|\Z)' (IGNORECASE | DOTALL)` extracts the content of the `###` Recommendation section from the LLM's markdown output. Lines are cleaned by stripping bullet prefixes (`-`, `*`, numbered lists) and filtered for length > 5 characters. Up to 5 action items are returned in `recommended_actions`.

9.7 Complete Response Schema

Field	Type	Description
question	string	The original user question
equipment_id	string null	Detected equipment ID, or null if not found in question
answer	string	Full markdown-formatted LLM response
confidence	float (0.0-0.95)	Heuristic confidence score based on evidence count
risk_category	string	Critical / High / Medium / Low (from answer keyword scan)
root_causes	list[string]	Identified root causes from taxonomy or incident history fallback
recommended_actions	list[string]	Extracted action items from Recommendation section (max 5)
sources.manuals	list[{manual,page}]	OEM manual citations with manual stem and page number
sources.relational	list[{type,data}]	PostgreSQL records used as context (type: incident/inspection/work_order)

Table 9.1 – /ask/copilot Response Schema

Chapter 10: Knowledge Gap Detection

10.1 Algorithm Overview

The Knowledge Gap Detection algorithm identifies equipment where a maintenance recommendation was made via inspection but no corrective action was completed, and a subsequent failure occurred. This 'gap' — recommended but not acted upon — represents a preventable failure for which operational evidence already existed.

10.2 Detection Criteria

An inspection is flagged as a knowledge gap if all three conditions are true simultaneously:

- `follow_up_required = 'Yes'` — the inspection explicitly required corrective follow-up.
- `risk_score >= 75` — the inspection risk assessment indicated High or Critical risk.
- Equipment has incident history — at least one incident exists for the same `equipment_id`.

10.3 Algorithm Flow

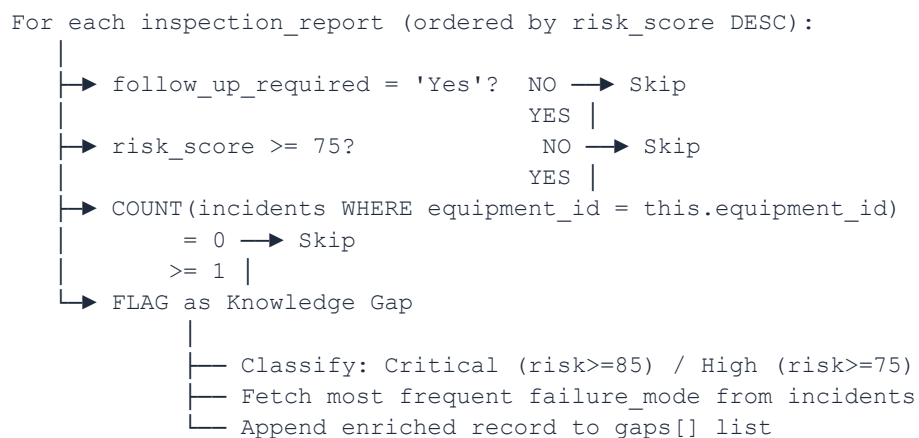


Figure 10.1 – Knowledge Gap Detection Algorithm Flow

10.4 SQL Implementation

The dashboard summary uses a single optimised SQL query with a correlated EXISTS subquery:

```

SELECT COUNT(DISTINCT ir.inspection_id)
FROM inspection_report ir
WHERE ir.follow_up_required = 'Yes'
      AND ir.risk_score >= 75
      AND EXISTS (
        SELECT 1 FROM incident_report inc
        WHERE inc.equipment_id = ir.equipment_id
      )
  
```

The `/insights/knowledge-gaps` endpoint uses Python-level iteration over inspection records with individual incident COUNT queries per qualifying record, enabling per-gap enrichment (most common failure mode, incident count, risk classification).

10.5 Business Value

The Knowledge Gap metric directly translates to business value. Every flagged gap represents a documented case where maintenance action was recommended, not taken, and the equipment subsequently failed. The preventable_failure_rate metric ($\text{gap_count} / \text{total_incidents} \times 100$) gives plant managers a single KPI to track improvement in inspection-to-action conversion over time. The top_assets list identifies the highest-priority equipment for immediate management attention.

Chapter 11: Machine Learning

11.1 Overview

AssetMind implements two independent machine learning pipelines. The AI4I pipeline predicts binary equipment failure probability using a Random Forest classifier. The CMAPSS pipeline predicts Remaining Useful Life (RUL) in engine cycles using an XGBoost regressor. Both follow the structure: dataset loading, feature engineering, StandardScaler normalisation, model training, evaluation, and pickle serialisation to backend/models/.

11.2 AI4I Failure Prediction

11.2.1 Dataset

UCI AI4I 2020 Predictive Maintenance dataset (ai4i2020.csv): 10,000 synthetic manufacturing process observations with 5 sensor features (Air temperature, Process temperature, Rotational speed, Torque, Tool wear) and a binary Machine failure target. Failure rate: approximately 3.4%, representing realistic industrial class imbalance.

11.2.2 Training Configuration

RandomForestClassifier: `n_estimators=200`, `max_depth=10`, `class_weight='balanced'` (compensates for 3.4% failure rate), `random_state=42`. Data split: 80% training / 20% test with stratification to preserve class ratio. StandardScaler applied on training set; transform only on test set. Both model and scaler saved as pickle files.

11.2.3 Inference

Five sensor values assembled into a numpy array, scaled with saved StandardScaler, fed to `predict_proba()`. Failure probability = class 1 probability. Risk thresholds: High (≥ 0.7), Medium (≥ 0.4), Low (< 0.4). Top 3 features by `feature_importances_` magnitude returned as `top_features`.

Feature	Physical Meaning	Unit
Air temperature [K]	Ambient air temperature at machine	Kelvin
Process temperature [K]	Process fluid / workpiece temperature	Kelvin
Rotational speed [rpm]	Spindle or rotor rotational speed	RPM
Torque [Nm]	Applied mechanical torque	Newton-metres
Tool wear [min]	Cumulative tool operating time	Minutes

Table 11.1 – AI4I Feature Set

11.3 CMAPSS RUL Prediction

11.3.1 Dataset

NASA CMAPSS FD001: 100 turbofan engine run-to-failure trajectories (single operating condition, single fault mode: HPC degradation). Training set: 20,631 cycles; test set: 13,096 cycles with ground-truth RUL in RUL_FD001.txt.

11.3.2 Feature Engineering

14 of 21 available CMAPSS sensor channels selected: s2 (fan speed), s3 (HPC outlet pressure), s4 (LPT outlet temperature), s7 (HPC static pressure), s8 (fuel-flow ratio), s9 (bypass ratio), s11 (HPC bleed enthalpy), s12 (LPT coolant bleed), s13 (HPT coolant bleed), s14 (LPT outlet pressure), s15 (bleed enthalpy), s17 (total pressure ratio), s20 (HPT exit temperature), s21 (LPT exit temperature). Near-constant sensors omitted.

RUL target: piecewise-linear formulation clipped at MAX_RUL=130 cycles. Healthy engines assigned a flat ceiling of 130; RUL decreases linearly from the degradation onset to 0 at end-of-life. This prevents the regressor from attempting to estimate very large RUL values for brand-new engines.

11.3.3 Training Configuration

XGBRegressor: n_estimators=300, max_depth=6, learning_rate=0.05, subsample=0.8, colsample_bytree=0.8, random_state=42. Slower learning rate (0.05) with 300 trees provides better generalisation. Subsample and colsample_bytree regularisation reduce variance on 14-feature input.

11.3.4 Health Score Mapping

health_score = min((rul_pred / 400) x 100, 100). Negative predictions floored at 0. Degradation trend thresholds: Healthy (RUL >= 300 cycles), Stable (>= 200), Declining (>= 100), Critical (< 100).

11.4 Model File Summary

File	Algorithm	Dataset	Purpose
backend/models/ai4i_model.pkl	RandomForestClassifier	UCI AI4I 2020	Predicts binary failure probability (0.0-1.0)
backend/models/ai4i_scaler.pkl	StandardScaler	UCI AI4I 2020	Normalises 5 AI4I sensor features for inference
backend/models/cmapss_model.pkl	XGBRegressor	NASA CMAPSS FD001	Predicts RUL in engine cycles (0-130+)
backend/models/cmapss_scaler.pkl	StandardScaler	NASA CMAPSS FD001	Normalises 14 CMAPSS sensor channels for inference

Table 11.2 – Trained Model Files

Chapter 12: API Documentation

12.1 Complete Endpoint Reference

Method	Endpoint	Description	Input / Body
GET	GET /	API health check + version 2.0.0	None
GET	GET /dashboard/	Aggregate fleet KPI statistics	None
GET	GET /dashboard/high-risk-assets	All assets ranked by composite risk score	None
GET	GET /equipment/	List all 25 equipment records	None
GET	GET /equipment/{id}	Single equipment record by ID	Path: equipment_id
GET	GET /equipment/{id}/briefing	Aggregated equipment summary card	Path: equipment_id
GET	GET /equipment/{id}/timeline	Chronological maintenance event timeline	Path: equipment_id
GET	GET /equipment/{id}/incidents	Incident history DESC	Path: equipment_id
GET	GET /equipment/{id}/inspections	Inspection history DESC	Path: equipment_id
GET	GET /equipment/{id}/health	Penalty-based health score 0-100	Path: equipment_id
GET	GET /insights/knowledge-gaps	Full knowledge gap analysis with per-gap detail	None
GET	GET /insights/knowledge-gaps/summary	Gap summary: count, rate, top assets	None
GET	GET /insights/executive	AI-derived executive insights from live data	None
POST	POST /ask/	Simple RAG over OEM manuals only	JSON: {question: string}
POST	POST /ask/copilot	Hybrid RAG + PostgreSQL + LLM copilot	JSON: {question: string}
POST	POST /predict/failure	Failure probability via Random Forest	JSON: 5 sensor floats
POST	POST /predict/rul	Remaining Useful Life via XGBoost	JSON: engine_id + 14 sensor values

Table 12.1 – Complete API Endpoint Reference (17 endpoints)

12.2 Example Requests and Responses

POST /ask/copilot — Request Body

```
{ "question": "Why is PMP-CW-101 failing?" }
```

POST /ask/copilot — Response (abbreviated)

```
{ "question": "Why is PMP-CW-101 failing?",
  "equipment_id": "PMP-CW-101",
  "answer": "### Summary\nElevated vibration and suction noise...",
  "confidence": 0.87, "risk_category": "High",
  "root_causes": ["Cavitation", "Bearing Wear"],
  "recommended_actions": ["Clean suction strainer", "Inspect bearings"],
  "sources": {
    "manuals": [{"manual": "1_CR", "page": 42}],
    "relational": [{"type": "incident", "data": {...}}] }}
```

POST /predict/failure — Request Body

```
{ "air_temperature": 298.0, "process_temperature": 308.0,
  "rotational_speed": 1400.0, "torque": 45.0, "tool_wear": 120.0 }
```

POST /predict/failure — Response

```
{ "failure_probability": 0.7823, "risk_level": "High",
  "top_features": ["Tool Wear", "Torque", "Rotational Speed"] }
```

POST /predict/rul — Request Body

```
{ "engine_id": 12, "sensor_values": [642.2, 1591.8, 1400.6,
  553.4, 2388.0, 9065.0, 47.2, 521.7, 2388.0, 8141.1, 8.42, 0.03, 392.0, 38.96]
}
```

POST /predict/rul — Response

```
{ "health_score": 68, "remaining_useful_life": 112, "degradation_trend":
  "Declining" }
```

12.3 Error Response Codes

HTTP Code	Condition	Response Body
404	Equipment ID not in database	{ "error": "Equipment not found" }
422	Invalid Pydantic schema / wrong types	{ "detail": [{"loc": "...", "msg": "...}] }
503	ML model pickle file not found	{ "detail": "Model not ready: ...path. Run train_ai4i.py first." }
500	Database / ChromaDB / internal error	{ "detail": "...exception message..." }

Table 12.2 – API Error Response Codes

Chapter 13: Application Workflow

13.1 End-to-End User Journey

- STEP 1: User opens browser → navigates to /
- └ Landing Page: Hero banner, Features, HowItWorks sections
- STEP 2: User clicks 'Launch App' → navigates to /app (Dashboard)
- └ Promise.all: getDashboardStats + getHighRiskAssets + getKnowledgeGapsSummary
 - └ Renders: KPI cards, fleet chart, risk donut, high-risk table (top 8), knowledge gap alerts
- STEP 3: User clicks asset row (e.g. PMP-CW-101)
- └ Navigate to /app/equipment/PMP-CW-101
 - └ Load: getEquipmentById + getEquipmentHealth + getBriefing + getEquipmentIncidents + getEquipmentInspections
 - └ View: health score, briefing panel, incident/inspection tables
- STEP 4: Engineer clicks 'View Timeline'
- └ getTimeline → sorted events (Inspection, WO, Incident, Recommendation, Repair)
- STEP 5: Engineer opens Copilot → types question
- └ 'Why is PMP-CW-101 failing?'
 - └ POST /ask/copilot:
 - (a) extract_id → 'PMP-CW-101'
 - (b) postgres_service → incidents + inspections + work orders
 - (c) chromadb.query → top-8 OEM manual chunks
 - (d) Sarvam 105B → structured markdown response
 - (e) confidence score + risk_category + root_causes computed
 - (f) ChatWindow renders rich evidence card
- STEP 6: Engineer runs ML Prediction (Equipment Profile)
- └ POST /predict/failure → failure_probability, risk_level, top_features
 - └ POST /predict/rul → health_score, RUL, degradation_trend
- STEP 7: Plant Manager reviews Insights page
- └ Promise.all: getExecutiveInsights + getKnowledgeGapsSummary + getHighRiskAssets + getKnowledgeGaps
 - └ View: executive card, risk trend chart, failure modes, predicted failures list, knowledge gap detail

Figure 13.1 – Complete End-to-End Application Workflow

Chapter 14: Folder Structure

14.1 Complete Project Directory Tree

AssetMind/	
├── README.md	Project overview and setup guide
├── assetmind.html	Static single-file HTML landing page
├── backend/	FastAPI Python application
│ ├── main.py	Env key mapping: SARVAM→GEMINI
│ └── app/	
│ ├── main.py	FastAPI app init, routers, CORS
│ ├── config.py	API key environment configuration
│ ├── db.py	SQLAlchemy engine + SessionLocal
│ └── routes/	
│ ├── dashboard.py	GET /dashboard/ and /high-risk-assets
│ ├── equipment.py	7 GET endpoints under /equipment
│ ├── insights.py	3 GET endpoints under /insights
│ ├── predict.py	POST /predict/failure + /predict/rul
│ └── rag.py	POST /ask/ + /ask/copilot
│ └── services/	
│ ├── postgres_service.py	All SQL queries and aggregates
│ ├── ml_service.py	Lazy model loading and inference
│ └── equipment_parser.py	Regex equipment ID extractor
│ └── ingestion/	
│ └── run_cognee_sync.py	(Legacy; unused in v2)
│ ├── db/	
│ ├── postgres.py	(Empty placeholder)
│ └── schemas.py	(Empty placeholder)
│ ├── manuals/	9 OEM PDF documents
│ ├── models/	Trained .pkl model artifacts
│ ├── ai4i_model.pkl	RandomForestClassifier
│ ├── ai4i_scaler.pkl	StandardScaler for AI4I
│ ├── cmapss_model.pkl	XGBRegressor
│ └── cmapss_scaler.pkl	StandardScaler for CMAPSS
│ ├── scripts/	
│ ├── train_ai4i.py	Train and save AI4I RF model
│ ├── train_cmapss.py	Train and save CMAPSS XGB model
│ ├── ingest_manuals.py	Manual ingestion helper script
│ ├── test_rag.py	RAG integration test script
│ └── test_collection.py	ChromaDB collection test script
│ ├── api/	(Legacy route stubs – unused in v2)
│ └── services/	(Legacy service stubs – unused in v2)
└── frontend/	React 19 + Vite 8 SPA
│ ├── package.json	npm dependencies and scripts
│ ├── vite.config.js	Vite + React plugin + TailwindCSS config
│ ├── index.html	Root HTML shell template
│ └── src/	
│ ├── App.jsx	BrowserRouter + Routes definition
│ ├── main.jsx	React 19 createRoot entry point
│ ├── index.css	Tailwind @import
│ ├── styles/global.css	Design system: CSS custom properties
│ └── components/	Reusable UI components (9 files)

└─ pages/	Application pages (7 files)
└─ Landing/	7-section marketing landing page
└─ services/api.js	Axios client + 16 exported API functions
└─ data/	
└─ equipment.csv	25 industrial assets
└─ work_orders.csv	500 maintenance work orders
└─ inspection_reports.csv	50 inspection records
└─ incident_reports.csv	100 failure incident records
└─ ai4i/ai4i2020.csv	UCI AI4I 2020 dataset (10,000 rows)
└─ cmapss/	NASA CMAPSS FD001 (train/test/RUL)
└─ rag/	
└─ ingestion_pipeline.py	PDF -> ChromaDB ingestion (run once)
└─ graph/	
└─ build_graph.py	Knowledge graph scaffold (future)

Figure 14.1 – Complete AssetMind Project Directory Tree

Chapter 15: Screenshots

The following placeholders mark the intended location of application screenshots to be captured from a running AssetMind instance. Each figure label describes the UI state and key visible elements.

15.1 Operations Dashboard

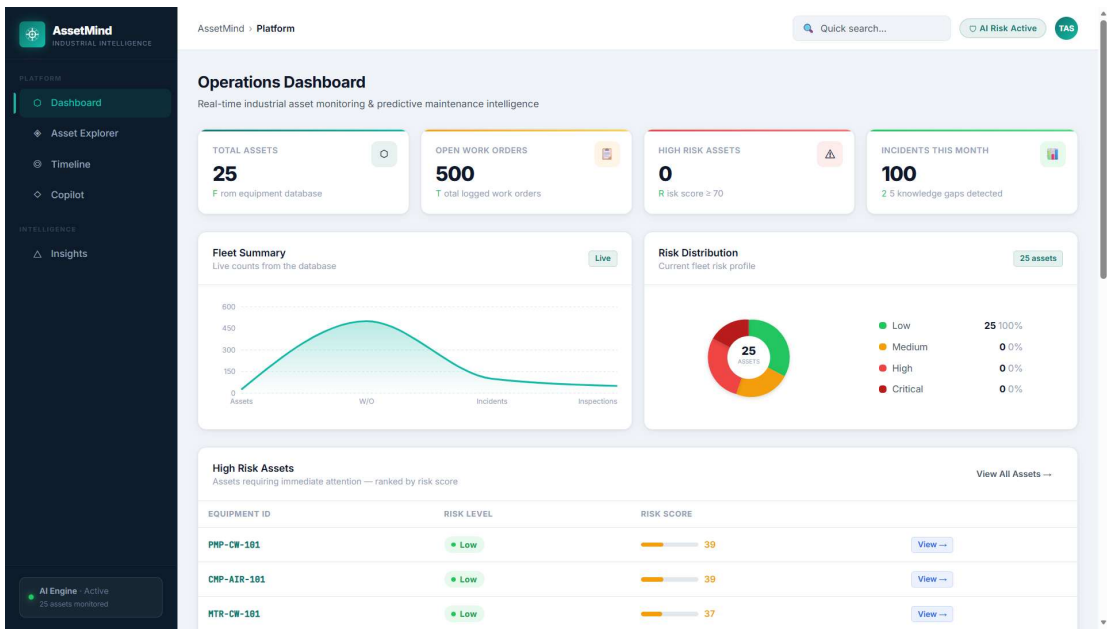


Figure 15.1 – Operations Dashboard with real-time PostgreSQL-backed KPIs, fleet summary AreaChart (teal gradient), risk distribution SVG donut, and AI-detected knowledge gap alerts.

15.2 Asset Explorer

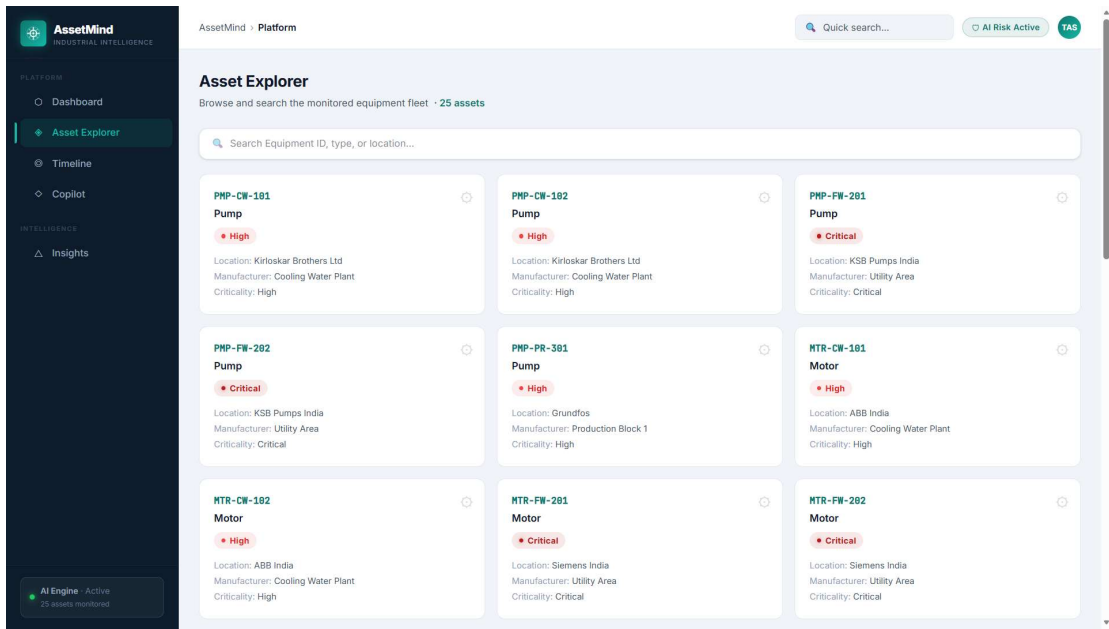


Figure 15.2 – Asset Explorer showing 25-asset card grid with RiskBadge, equipment type emoji watermarks, and real-time search filtering by ID, type, or location.

15.3 Equipment Profile

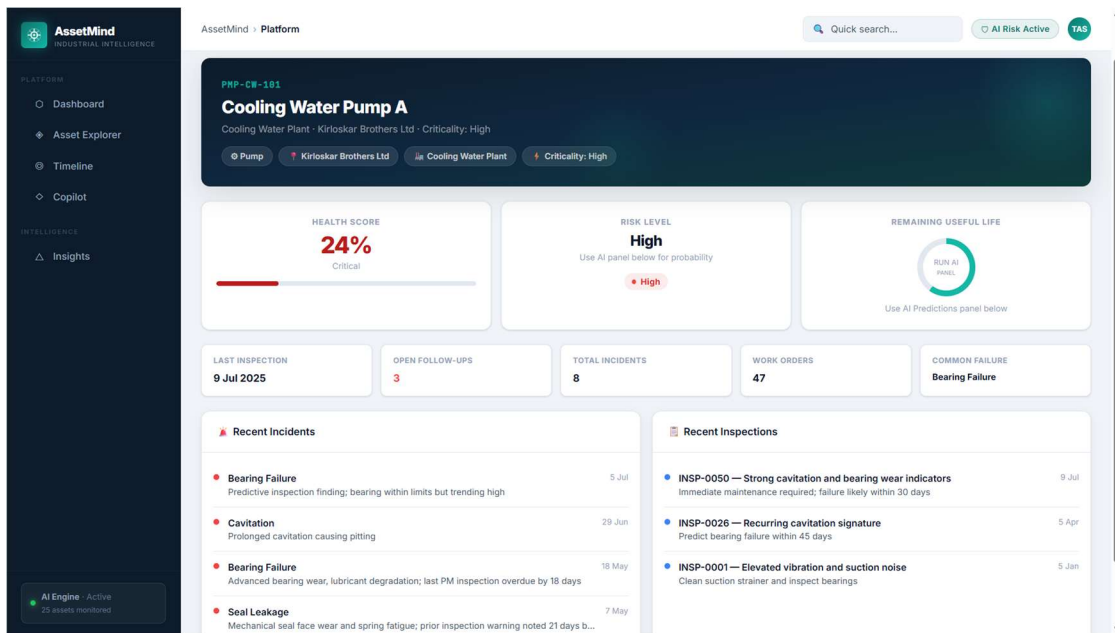


Figure 15.3 – Equipment Profile for PMP-CW-101 showing health score (colour-coded gauge), briefing summary, incident and inspection history tables, and interactive ML prediction panels.

15.4 Timeline View

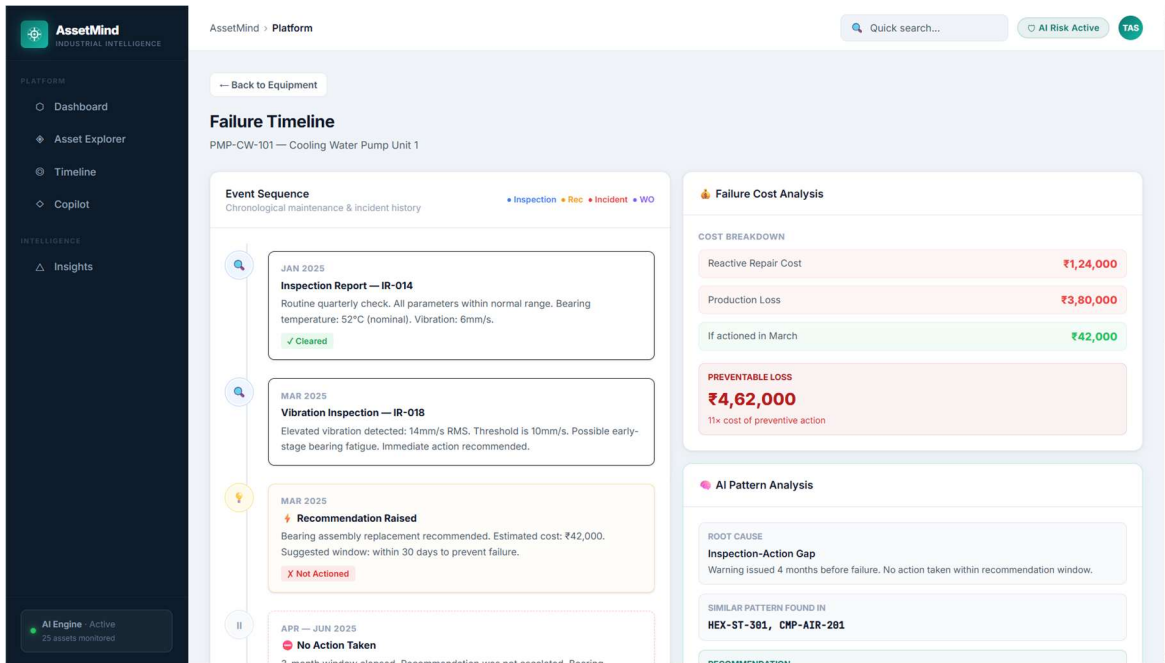


Figure 15.4 – Maintenance Timeline displaying colour-coded events (Inspection, Recommendation, Work Order, Repair, Incident) in ascending date order.

15.5 AI Copilot

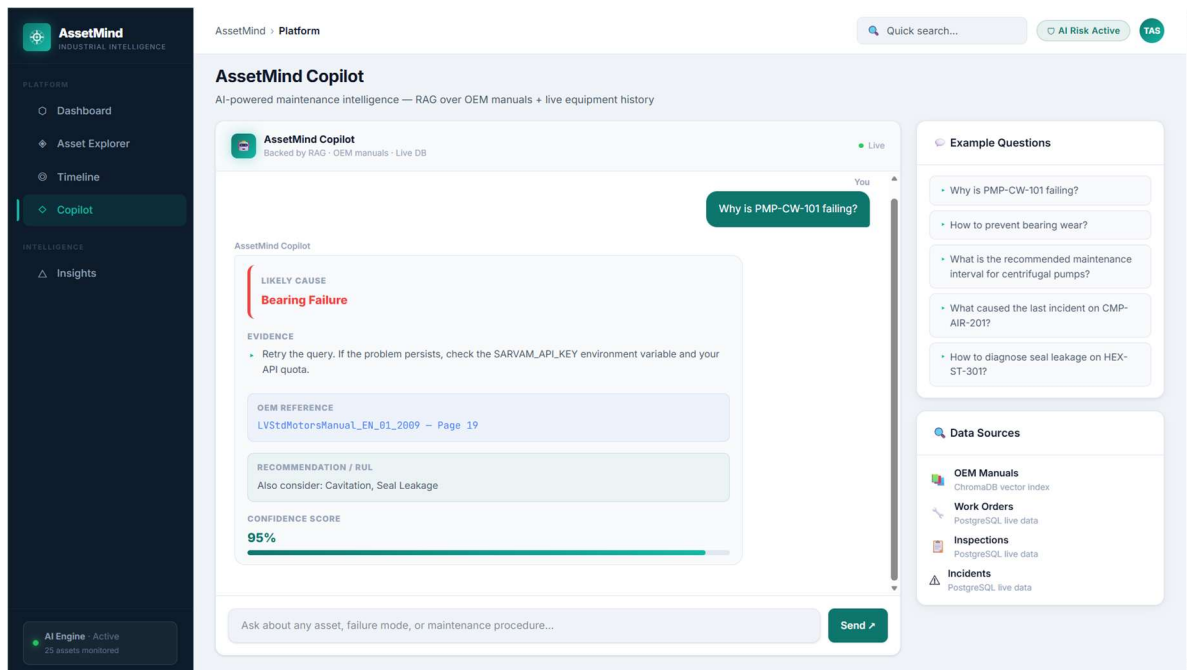


Figure 15.5 – AI Copilot chat interface showing a structured response card with 87% confidence, High risk category, root causes (Cavitation, Bearing Wear), evidence bullets, and OEM manual citation (CR Manual, Page 42).

15.6 ML Prediction Panel

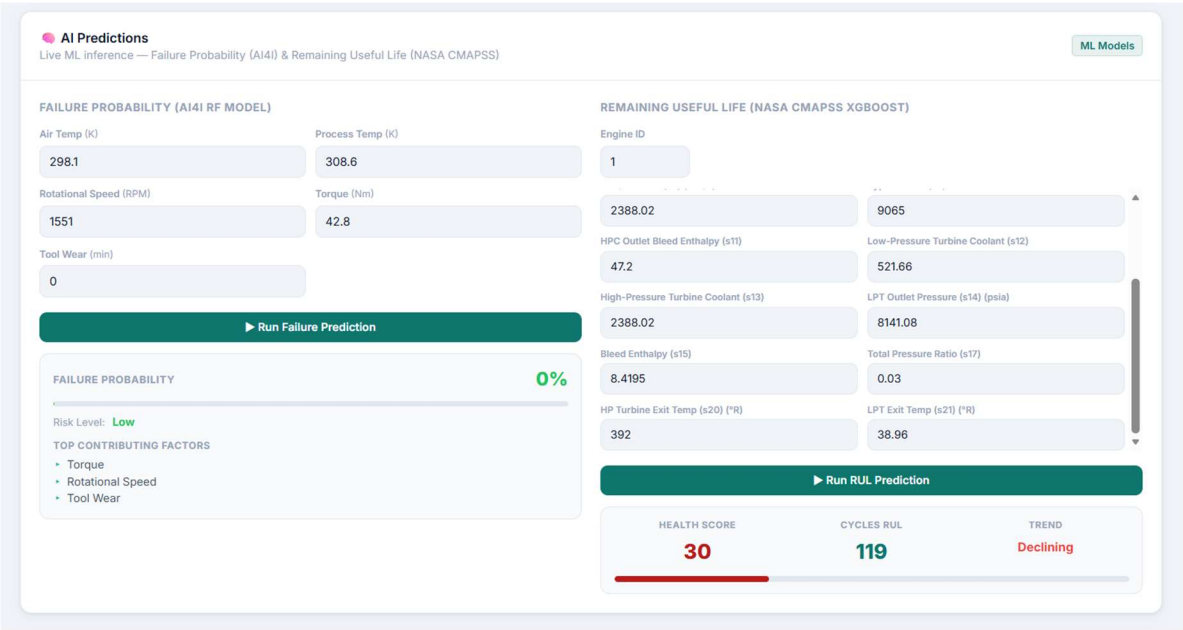


Figure 15.6 – Equipment Profile ML Prediction Panel showing failure probability (0.7823, High risk, top features: Tool Wear, Torque, Rotational Speed) and RUL prediction (112 cycles, Declining, health_score: 68%).

15.7 Insights & Analytics

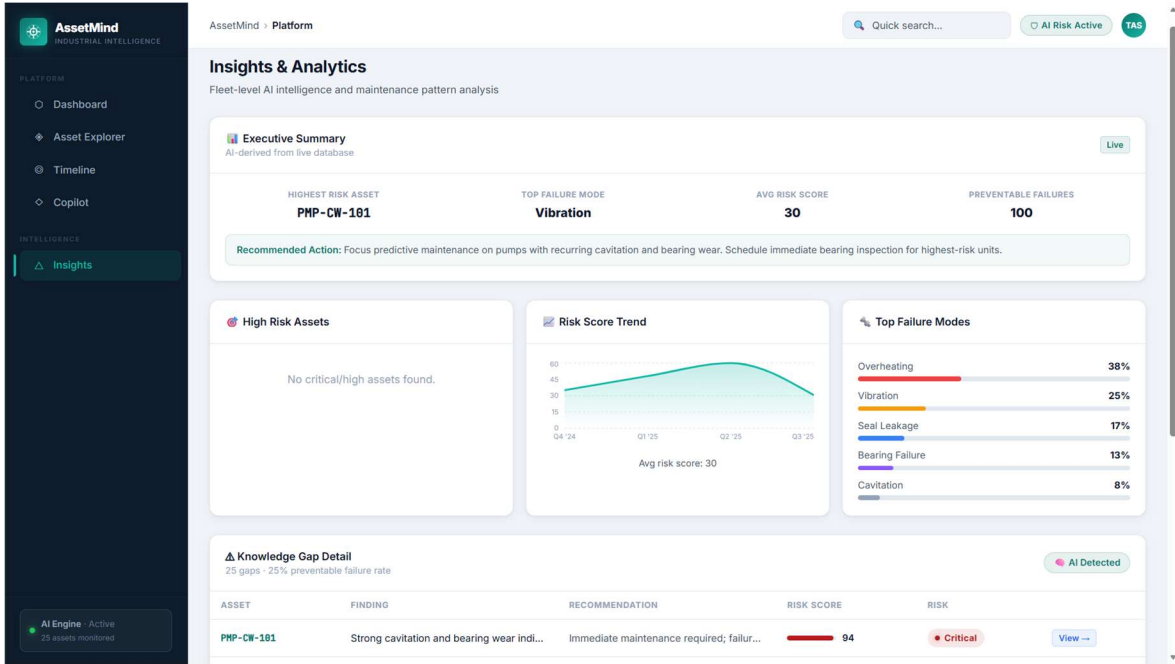


Figure 15.7 – Insights page showing executive AI insights, fleet risk trend AreaChart, failure mode frequency bar chart (from knowledge gap aggregation), and top predicted failures ranked by composite risk score.

Chapter 16: Challenges Faced

16.1 ChromaDB API Version Compatibility

ChromaDB's PersistentClient API changed significantly between versions, with incompatible constructor arguments and collection management methods between minor releases. The ingestion pipeline and the RAG route initially used inconsistent API patterns. Resolution required aligning both files to use the same PersistentClient + get_or_create_collection / get_collection pattern and pinning the chromadb package version.

16.2 Sarvam AI SDK Key Mapping

The sarvamai Python SDK expects SARVAM_API_KEY but underlying model libraries were configured to look for GEMINI_API_KEY, suggesting the SDK wraps a Gemini-compatible endpoint. A compatibility shim was implemented in backend/main.py that reads SARVAM_API_KEY and writes it to GEMINI_API_KEY on process startup if the latter is unset, preserving compatibility with both environments.

16.3 Empty LLM Response Handling

The Sarvam AI API occasionally returns an empty or None content field in the response object (choices[0].message.content = None). This caused unhandled NoneType dereferences in early implementations. The fix adds .strip() and empty-string checks after extraction, replacing empty responses with a structured fallback message instructing the user to retry and check API quota.

16.4 CMAPSS RUL Target Engineering

The piecewise-linear RUL target for CMAPSS required careful implementation. The naive approach (max_cycle - current_cycle) produces RUL values of 350+ for early cycles, making the regression task unnecessarily difficult. The MAX_RUL=130 clip and the ground-truth-based piecewise formulation for the test set (using RUL_FD001.txt as the terminal value and backfilling for earlier cycles) required debugging against the dataset specification PDF included in data/cmapss/.

16.5 AI4I Class Imbalance

The AI4I dataset's 3.4% positive (failure) rate creates severe class imbalance. Without correction, a naive classifier achieves 96.6% accuracy by predicting 'No Failure' for all inputs. The class_weight='balanced' parameter in RandomForestClassifier automatically weights the minority class inversely proportional to its frequency. Combined with stratified train-test splitting, this ensures the model learns meaningful failure patterns rather than the majority-class prior.

16.6 RAG Context Window Management

Early Copilot implementations concatenated all retrieved context without length limits, occasionally exceeding the LLM's token budget. The fix imposes explicit character limits: 4,000 characters for the simple /ask/ endpoint (manuals only) and 12,000 characters for /ask/copilot (relational + manuals). These limits balance context richness against token budget and response latency.

16.7 Equipment ID Regex Precision

Extracting equipment IDs from free-text questions required a regex that matches the AssetMind ID convention (prefix-location-number, e.g. PMP-CW-101) without false-positive matches on common English tokens. The lookahead (`(?=[A-Z0-9-]*\d)`) requiring at least one digit, combined with a minimum length of 3 characters, achieves this. Early patterns without the digit requirement matched words like 'PMP' as IDs.

16.8 Frontend Multi-Call State Management

The EquipmentProfile page makes six concurrent API calls. Managing independent loading, error, and data states for each call without a global state manager required careful useState structuring. Independent calls (equipment detail, health, briefing, incidents, inspections) are parallelised with Promise.all; ML prediction calls (failure and RUL) are triggered separately on user action to avoid blocking the initial page load.

Chapter 17: Future Enhancements

17.1 IoT Real-Time Sensor Integration (MQTT / OPC-UA)

The current platform ingests historical maintenance records. A natural evolution is to connect directly to industrial IoT sensor networks via MQTT brokers or OPC-UA servers. Real-time sensor streams (vibration, temperature, pressure) would feed the ML prediction endpoints continuously, enabling live degradation monitoring rather than batch on-demand prediction. The existing `/predict/failure` and `/predict/rul` endpoints are already architected for low-latency inference.

17.2 Digital Twin Integration

A digital twin layer would create virtual representations of physical assets, continuously updated from sensor feeds. Simulation-driven health projections could overlay the Timeline view, giving engineers a forward-looking degradation trajectory alongside the current backward-looking maintenance history.

17.3 SAP Plant Maintenance Integration

Many large industrial organisations use SAP PM as their CMMS. An integration layer synchronising SAP PM work orders and notification history into AssetMind's PostgreSQL would eliminate manual data loading and allow the platform to operate as an AI intelligence layer on top of existing SAP investments without requiring CMMS replacement.

17.4 Multi-Agent Maintenance Orchestration

The current Copilot is a single-turn question-answering system. A multi-agent architecture using CrewAI or LangGraph could enable an autonomous diagnostic workflow: a Planner agent decomposes the maintenance question, a Data Retrieval agent queries PostgreSQL and ChromaDB in parallel, a Reasoning agent synthesises findings, and a Report agent formats the final recommendation — all coordinated without manual steering.

17.5 Edge AI Deployment

For industrial facilities with intermittent connectivity, both the RandomForest and XGBoost models are lightweight enough for edge deployment using ONNX or pickle + a minimal Python runtime. Edge inference at the PLC or edge gateway level enables real-time failure scoring without cloud round-trips, critical for latency-sensitive process control environments.

17.6 Predictive Maintenance Scheduling

An automated scheduler using RUL predictions could generate proactive work orders before equipment reaches critical degradation. When an asset's predicted RUL drops below a configurable threshold, the system automatically creates a draft work order — closing the loop from ML prediction to actionable maintenance task.

17.7 Authentication and Multi-Tenant Support

The current version has no authentication — the Login page is UI-only. Production deployment requires OAuth2/JWT authentication, role-based access control (engineer vs. manager vs. admin), and multi-tenant isolation for organisations deploying AssetMind across multiple plants or business units. The FastAPI dependency injection system is well-suited for adding OAuth2 middleware.

17.8 Voice Interface for Shop Floor Use

A voice interface using Whisper (speech-to-text) and a TTS engine would enable maintenance engineers on the shop floor to query the Copilot hands-free during physical maintenance activities. 'What is the torque specification for the Grundfos CR pump shaft seal?' could be answered immediately without the engineer stopping to type.

17.9 Expanded OEM Knowledge Base

The current corpus covers 9 manuals for 3 equipment families. The ingestion pipeline scales to hundreds of documents without code changes. Automatic ingestion triggers on new PDF uploads (e.g. via a watched S3 bucket) would keep the knowledge base current. Coverage expansion to all equipment types in the monitored fleet would significantly improve Copilot quality for less-common assets.

Chapter 18: Conclusion

AssetMind demonstrates that Retrieval-Augmented Generation, structured SQL analytics, and machine learning prediction can be composed into a coherent, production-quality industrial intelligence platform within a modern full-stack architecture. The platform successfully addresses the core maintenance intelligence problem: fragmented data and manual documentation cannot be effectively utilised by maintenance engineers under operational pressure.

The key technical achievements of AssetMind v2.0 are: a functioning RAG pipeline over 9 real OEM technical manuals (Grundfos, Atlas Copco, Siemens) enabling natural-language queries grounded in authoritative documentation; a Knowledge Gap Detection algorithm that correlates inspection recommendations with incident outcomes to quantify preventable failure rates as a business KPI; two trained ML models (RandomForest on UCI AI4I 2020, XGBoost on NASA CMAPSS FD001) deployed through production-ready FastAPI endpoints with lazy loading; a hybrid AI Copilot combining PostgreSQL relational context and ChromaDB vector retrieval with evidence-weighted confidence scoring; and a professional React 19 SPA with six application pages, a teal design system, and real-time loading states.

The platform exhibits strong scalability characteristics: the PostgreSQL schema and query patterns scale linearly with asset count; the ChromaDB vector index scales to thousands of document chunks without code changes; the ML models are stateless, deployable behind load balancers for high-availability serving.

AssetMind is directly applicable to manufacturing plants, utility facilities, refineries, pharmaceutical production sites, and any industrial environment where equipment reliability is operationally and financially critical. The platform's design — prioritising evidence-backed responses, audit trails via source citations, and explainable confidence scoring — aligns with the operational culture of safety-conscious industries where engineers must trust and verify AI recommendations before acting.

AssetMind represents a compelling example of applied AI engineering: taking proven technologies (LLMs, vector databases, classical ML, modern frontend frameworks) and composing them into a system that solves a clearly defined industrial problem with measurable business impact — fewer preventable failures, faster root cause diagnosis, and better utilisation of existing maintenance knowledge.

References

1. Matzka, S. (2020). AI4I 2020 Predictive Maintenance Dataset. UCI Machine Learning Repository. <https://doi.org/10.24432/C5HS5C>
2. Saxena, A., Goebel, K., Simon, D., & Eklund, N. (2008). Damage Propagation Modeling for Aircraft Engine Run-to-Failure Simulation. IEEE International Conference on Prognostics and Health Management. NASA Ames Research Center.
3. Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Advances in Neural Information Processing Systems (NeurIPS 2020).
4. Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. ACM SIGKDD International Conference on Knowledge Discovery and Data Mining.
5. Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5-32.
6. Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. Proceedings of EMNLP 2019.
7. FastAPI Documentation. Sebastián Ramírez. <https://fastapi.tiangolo.com>
8. React Documentation. Meta Platforms. <https://react.dev>
9. ChromaDB Documentation. Chroma Core. <https://docs.trychroma.com>
10. Grundfos. (2023). CR, CRI, CRN, CRT Installation and Operation Manual (98419736). Grundfos Group.
11. Atlas Copco. GA 30 / GA 55 / GA 75 Air Compressor Service Manual. Atlas Copco N.V.
12. Siemens AG. (2021). SIMOTICS SD Low-Voltage Motors – Operating Instructions (A5E41126458A). Siemens AG.
13. Vite Documentation. Evan You et al. <https://vitejs.dev>
14. Tailwind CSS Documentation. Tailwind Labs. <https://tailwindcss.com>

Appendix A: Installation Guide

A.1 Prerequisites

- Python 3.10+ with pip
- Node.js 18+ with npm
- PostgreSQL 15 (local)
- Git
- SARVAM_API_KEY from Sarvam AI

A.2 Backend Setup

1. Clone: `git clone https://github.com/tasneem38/AssetMind.git && cd AssetMind`
2. Install Python deps: `pip install fastapi uvicorn sqlalchemy pycpg2-binary chromadb sentence-transformers langchain-text-splitters pypdf sarvamai scikit-learn xgboost pandas numpy python-dotenv`
3. Create PostgreSQL database: `psql -U postgres -c 'CREATE DATABASE assetmind;'`
4. Load CSV data into the four tables (equipment, work_order, inspection_report, incident_report) using COPY or your preferred import tool.
5. Ingest OEM manuals: `cd rag && python ingestion_pipeline.py`
6. Train AI4I model: `cd backend && python -m scripts.train_ai4i`
7. Train CMAPSS model: `python -m scripts.train_cmapss`
8. Set environment: `export SARVAM_API_KEY=<your_key>`
9. Start API: `cd backend && uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload`

A.3 Frontend Setup

11. Navigate: `cd frontend`
12. Install: `npm install`
13. Dev server: `npm run dev`
14. Open browser at `http://localhost:5173`

Appendix B: Environment Variables

Variable	Required	Description
SARVAM_API_KEY	Yes	Sarvam AI API subscription key for LLM access (sarvam-105b)
DATABASE_URL	Optional	PostgreSQL connection string (default: postgresql://postgres:mypassword@localhost:5432/assetmind)
GEMINI_API_KEY	Auto-set	Automatically mapped from SARVAM_API_KEY by backend/main.py on startup

Table B.1 – Environment Variables

Appendix C: Data Asset Summary

Dataset	Source	Size	Purpose
equipment.csv	Synthetic (AssetMind)	25 rows	Asset register: 9 equipment types, 4 plant locations
work_orders.csv	Synthetic (AssetMind)	500 rows	Maintenance work orders with embedded analytical patterns
inspection_reports.csv	Synthetic (AssetMind)	50 rows	Periodic inspections with vibration, temperature, risk scores
incident_reports.csv	Synthetic (AssetMind)	100 rows	Equipment failure incidents with root causes and preventability
ai4i2020.csv	UCI ML Repository	10,000 rows	AI4I 2020 Predictive Maintenance classification dataset
train_FD001.txt	NASA CMAPSS	20,631 cycles	Turbofan engine run-to-failure training trajectories
test_FD001.txt	NASA CMAPSS	13,096 cycles	Turbofan engine test cycles with ground-truth RUL
OEM Manuals (9 PDFs)	Grundfos/Copco/Siemens	~500 pages	Equipment-specific installation, maintenance documentation

Table C.1 – Complete Data Asset Summary

Appendix D: API Quick Reference

Endpoint	Method	Key Parameters
/dashboard/	GET	None
/dashboard/high-risk-assets	GET	None
/equipment/	GET	None
/equipment/{id}	GET	equipment_id: string
/equipment/{id}/briefing	GET	equipment_id: string
/equipment/{id}/timeline	GET	equipment_id: string
/equipment/{id}/health	GET	equipment_id: string
/insights/knowledge-gaps	GET	None
/insights/executive	GET	None
/ask/	POST	body: {question: string}
/ask/copilot	POST	body: {question: string}
/predict/failure	POST	body: {air_temperature, process_temperature, rotational_speed, torque, tool_wear}
/predict/rul	POST	body: {engine_id: int, sensor_values: list[float] (len=14)}

Table D.1 – API Quick Reference